Section 1: Introduction

Mushroom poisoning indicates a significant real-world risk, and visual inspection alone is often unreliable even for experienced individuals. This project aims to develop a machine learning based classification system to predict whether a mushroom is edible (e) or poisonous (p) using observable physical characteristics such as cap size and shape, gill attachment and color, stem dimensions, surface texture, and spore print color. The primary motivation of this work is to demonstrate how data-driven models can learn complex patterns from labeled examples and provide a systematic approach to mushroom edibility classification.

To achieve this objective, exploratory data analysis (EDA) is performed to examine class distribution and identify important feature relationships. Appropriate preprocessing techniques are applied to handle missing values without discarding significant portions of the dataset, followed by categorical encoding and feature scaling. Multiple supervised learning models—including K-Nearest Neighbors (KNN), Decision Tree, and a Neural Network are trained and evaluated using standard classification metrics such as accuracy, precision, recall, F1-score, confusion matrix, and ROC–AUC. Model performance is compared both before and after feature scaling to assess its impact, enabling informed model selection and providing a clear, reproducible workflow from raw data to final evaluation.

Section 2: Dataset Description

The dataset contains **61,069 observations** (rows) with 21 total columns, where 20 columns represent input features and one column (class) is the target label. Since the target variable is categorical with two possible classes (edible (e)and poisonous (p)) this problem is formulated as a binary classification task. It is not a regression problem because the output class represents discrete labels, rather than a numerical value.

The dataset includes a combination of quantitative and categorical features. Specifically, there are **three numerical attributes** (**cap-diameter, stem-height, and stem-width**) and **seventeen categorical attributes** describing mushroom characteristics such as cap shape, surface, and color; gill properties; stem properties; veil and ring attributes; spore-print color; and habitat.

A notable characteristic of this dataset is the presence of missing values in several categorical features, including *veil-type, spore-print-color, veil-color, stem-root,* and *stem-surface*. Removing rows with missing entries would significantly reduce the dataset size. So, **missing values were treated as a separate valid category during preprocessing**. This strategy preserves the full dataset and allows the models to learn whether the absence of information itself carries predictive significance.

Since the majority of input features are categorical, one-hot encoding was applied to convert categorical variables into a numerical representation suitable for machine learning algorithms. After encoding, the dataset becomes fully numeric and appropriate for training classification models. Additionally, numerical features can be scaled to support distance-based and gradient-based models, enabling fair comparison across different learning approaches.

Correlation

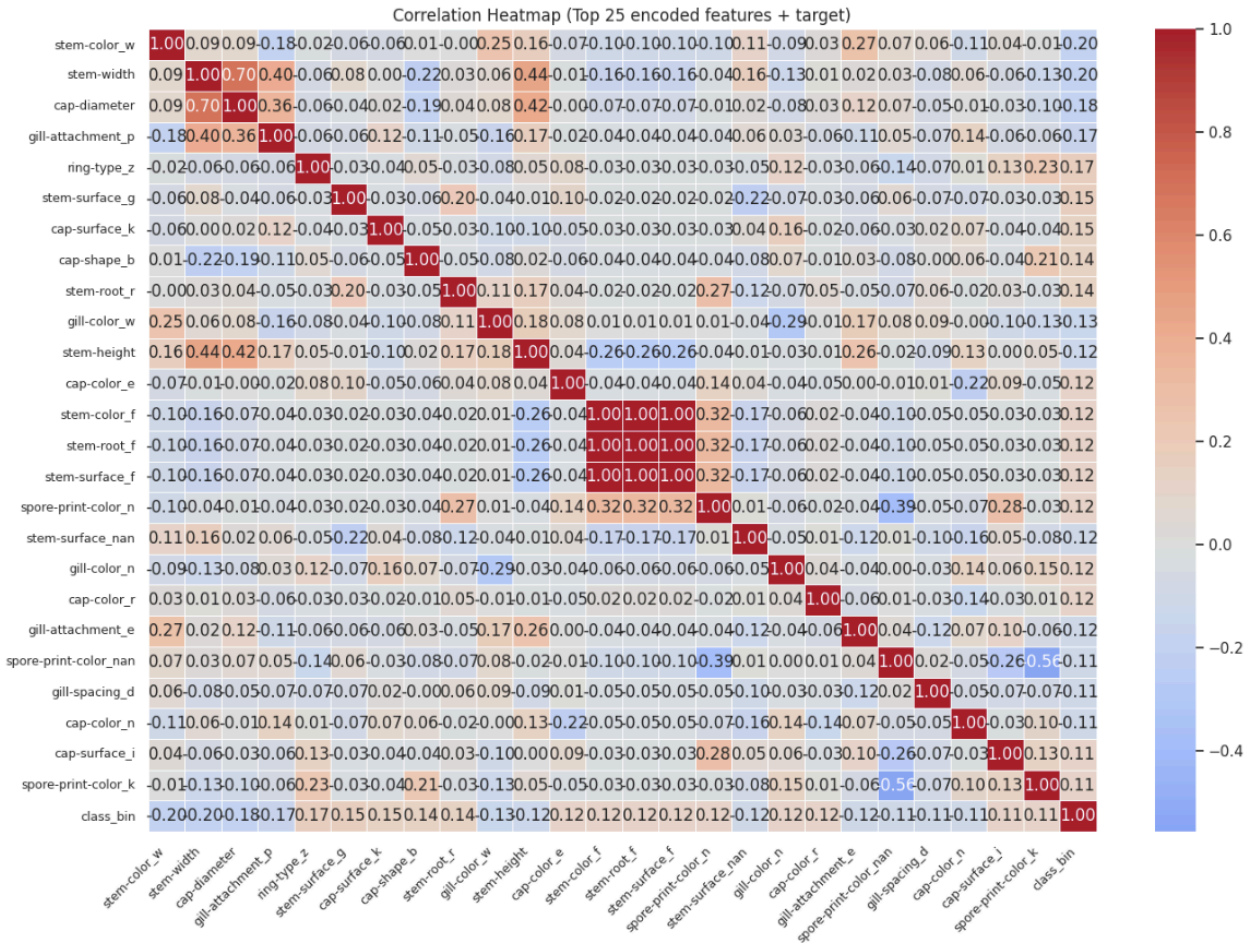


Figure 1: Correlation Heatmap

To perform correlation analysis, all categorical features were converted into numerical form using one-hot encoding, and the target label was mapped to a binary variable (class_bin) to allow computation of Pearson correlation coefficients. A correlation heatmap of the top 25 encoded features and the target was generated, where values range from −1 to 1. Here, values near zero indicate weak linear relationships.

The resulting heatmap shows that most off-diagonal correlations are close to zero, which indicates limited linear dependence among features. Several negative correlations are observed, which is expected in one-hot encoded data because categories within the same original feature are mutually exclusive. With respect to the target variable, the absolute correlations are relatively small ($|r| \leq 0.20$), indicating that no single feature exhibits a strong linear relationship with mushroom edibility. The strongest negative correlations with the target include stem-color_w ($r \approx -0.20$), stem-width ($r \approx -0.20$), and cap-diameter ($r \approx -0.18$), while the strongest positive correlations include ring-type_z ($r \approx +0.17$) and stem-surface_g ($r \approx +0.16$).

Overall, these results suggest that mushroom edibility is influenced by **combinations of features rather than any single attribute**, motivating the use of non-linear models.

Imbalanced dataset (Class distribution)



Figure 2: Class Distribution (Imbalance Check)

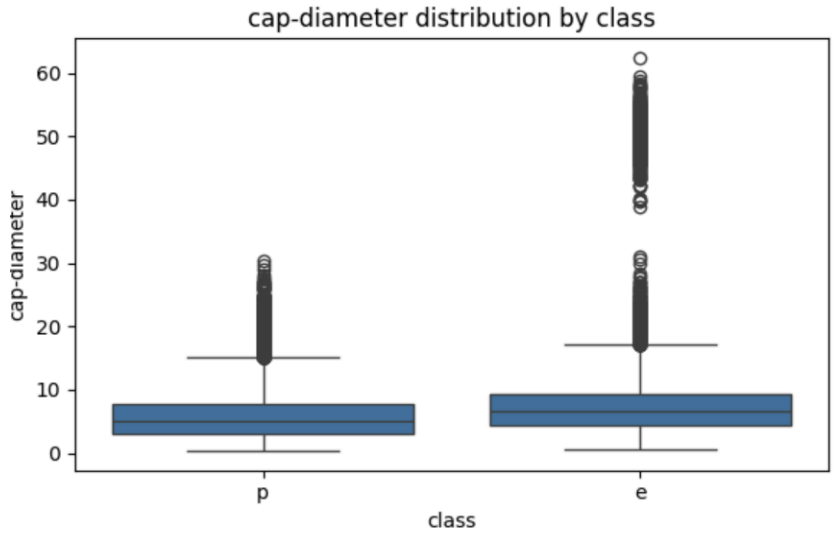
The target variable (class) consists of two categories: **edible (e)** and **poisonous (p)**. As shown in Figure 2, the dataset contains **61,069 samples**, of which **33,888 (≈55.5%)** are poisonous and **27,181 (≈44.5%)** are edible. This indicates a **mild class imbalance**. Poisonous mushrooms form the majority class.

Although the dataset is not severely imbalanced, this skew highlights the importance of evaluating models using metrics beyond accuracy. Therefore, precision, recall, F1-score, confusion matrix, and ROC–AUC are reported to ensure that model performance is robust for both classes and not biased toward the majority class.

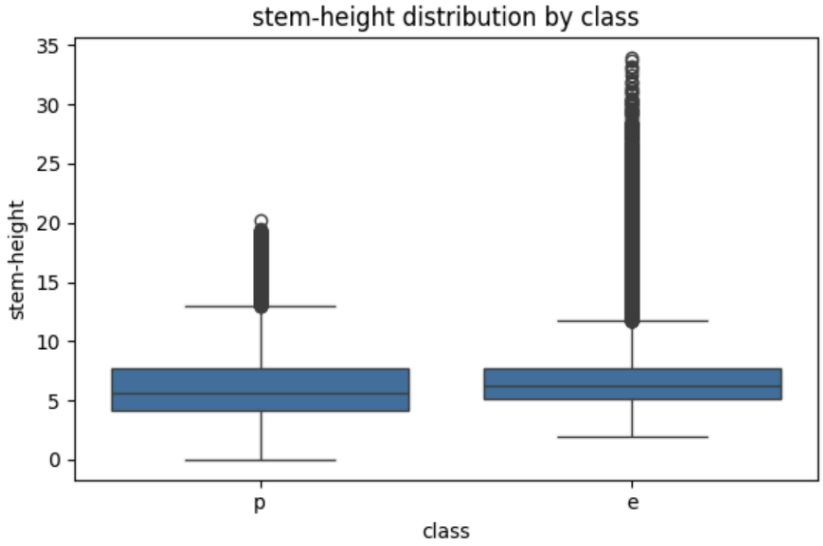
Exploratory Data Analysis (EDA)

EDA 1: Distribution of Numerical Features by Class

(a)



(b)



(c)

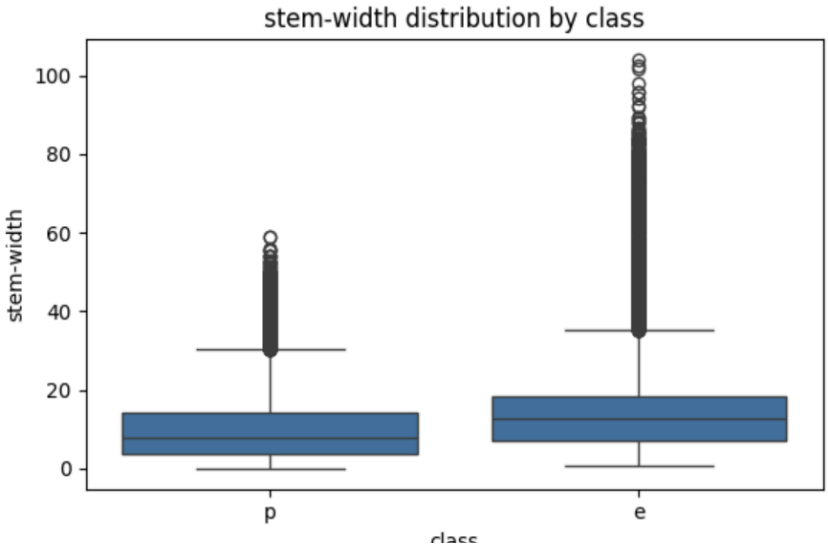


Figure 3 : (a) cap-diameter distribution by class
(b) stem-height distribution by class
(c) stem-width distribution by class

To examine how numerical attributes vary across classes, the distributions of **cap-diameter**, **stem-height**, and **stem-width** were compared between edible (e) and poisonous (p) mushrooms using boxplots (Figure 3). Across all three features,

the edible class exhibits a consistent upward shift in median and spread. This indicates generally larger physical dimensions. Among these, **stem-width and cap-diameter show the clearest separation**, suggesting they provide useful discriminative signals. However, noticeable overlap and outliers remain which confirms that no single numeric feature is sufficient for perfect class separation on its own.

EDA 2: Season vs Class Distribution

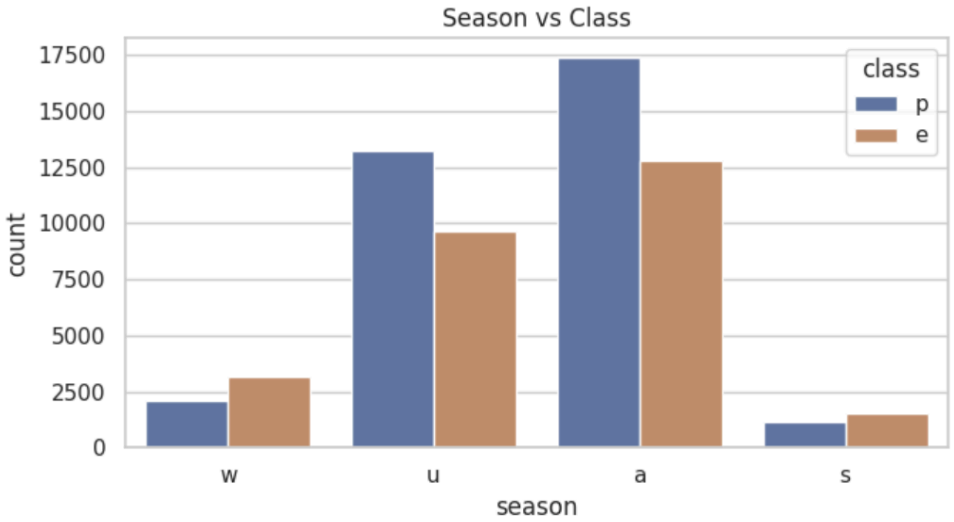


Figure 4. Season vs Class (Countplot)

The relationship between the categorical feature **season** and the target class was explored using a grouped count plot (Figure 4). Most observations occur in the *a* and *u* categories, while *w* and *s* contain fewer samples. The *a* and *u* categories show a higher proportion of poisonous mushrooms, whereas *w* and *s* are relatively more edible-dominant. This indicates that **season is associated with variations in class distribution** and may contribute predictive value when combined with other features.

EDA 3: Missing Value Analysis

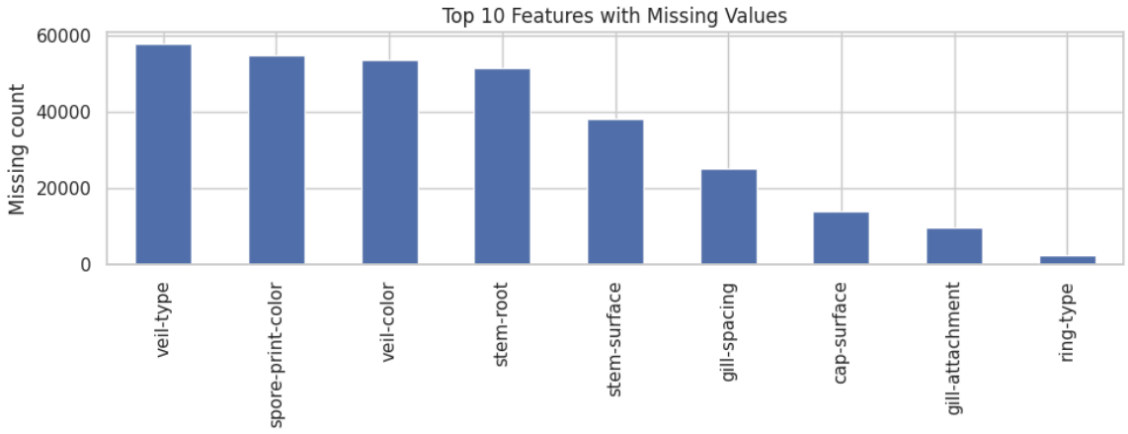


Figure: 5. Top 10 Features with missing values

A missing-value analysis was performed to understand data completeness and guide preprocessing decisions. The bar chart highlights that missing entries are concentrated in a small subset of categorical features. The highest missing counts occur in **veil-type (57,892)**, **spore-print-color (54,715)**, **veil-color (53,656)**, and **stem-root (51,538)**, followed by **stem-surface (38,124)** and **gill-spacing (25,063)**. Several other features such as **cap-surface (14,120)**, **gill-attachment (9,884)**, and **ring-type (2,471)** also contain missing values, while some columns (e.g., **cap-color**) show no missing entries. This distribution indicates that missingness is not uniformly spread across the dataset, so preprocessing must explicitly account for missing categories rather than assuming complete observations.

EDA 4: Categorical Feature Example — Does-Bruise-or-Bleed

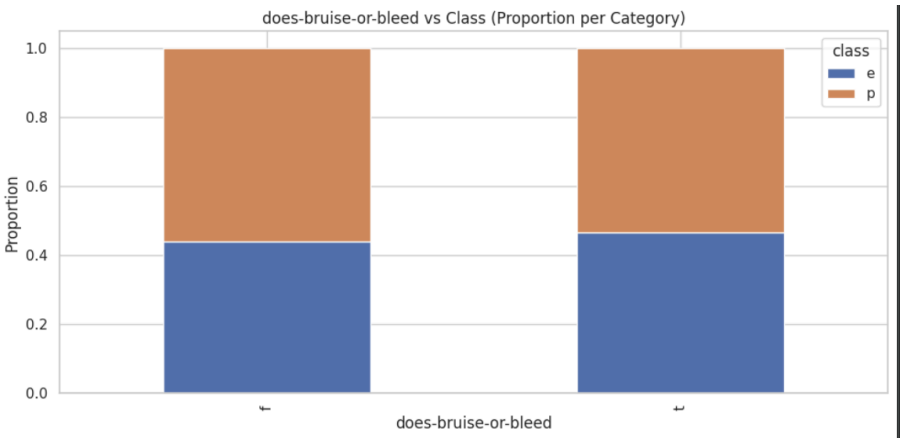


Figure 6. Does-bruise-or-bleed vs class

For the categorical EDA, the feature **does-bruise-or-bleed** was compared against the target **class** using a **proportion (stacked) bar chart**. The results show that both categories are **poison-dominant**, with only a small difference between them. When **does-bruise-or-bleed = f**, the dataset contains **22,239 edible** and **28,240 poisonous** samples, giving a poisonous rate of **55.94%**. When **does-bruise-or-bleed = t**, there are **4,942 edible** and **5,648 poisonous** samples, corresponding to a poisonous rate of **53.33%**. Overall, this feature provides **limited class separation** on its own (only 2.6 percentage-point difference. But it still contributes a useful signal when combined with other features in the classification models.

Section 3: Dataset Pre-processing

A) Null / Missing Values

Fault : The mushroom dataset contains a large amount of missing data across several categorical columns (e.g., veil-type, spore-print-color, veil-color, stem-root, etc.). This is problematic because most machine learning models cannot train on raw NaN values, and dropping all rows with missing entries would discard a substantial portion of the dataset.

Evidence (show cause): The "Top 10 Features with Missing Values" bar chart (EDA 3) clearly shows which columns have the highest missing counts, and the preprocessing log reports that the dataset initially contained **307,463** missing values in total.

Solution :

- For **numeric features**, missing values were imputed using the **median** of each numeric column (median imputation). Median was preferred over mean because it is more robust to outliers and skewed distributions.
- For **categorical features**, missing values were imputed using a constant placeholder category (**"missing"**). This approach preserves the information that a value was absent (instead of forcing an incorrect "average" value, which is not meaningful for categories).
  After imputation, the total missing values dropped from **307,463 to 0.**

```
Total missing values BEFORE: 307463
Total missing values AFTER: 0
```

Figure 7: missing value comparison

B) Categorical Values

Fault : Most input columns are categorical (string/object type). Many ML algorithms require numeric input and cannot directly interpret labels such as cap-shape, gill-color, ring-type, etc. Treating categories as raw integers would also create a false ordering (e.g., category "2" being "greater" than category "1"), which is incorrect.

Evidence : Before encoding, the feature matrix contained **20 input features** (shape **(61069, 20)**).

Solution : One-hot encoding was applied using pd.get_dummies(...), converting each category level into its own binary indicator column. After encoding, the feature space expanded to **128 columns** (shape **(61069, 128)**). This ensures categorical information is represented numerically without introducing artificial ordinal relationships.

```
Shape before encoding: (61069, 20)
Shape after encoding: (61069, 128)
```

Figure 8: Feature dimensionality before and after one-hot encoding

C) Feature Scaling

Fault : After encoding, the dataset contains a mix of **continuous numeric variables** (e.g., cap-diameter, stem-height, stem-width) and many **binary one-hot columns**. Models that depend on distance or gradient-based optimization (e.g., KNN, Logistic Regression, Neural Networks) can be affected when features are on different scales, because larger-scale variables may dominate learning.

Solution : The dataset was split first, then **Standard Scaling (z-score normalization)** was applied using StandardScaler. The scaler was **fit only on the training set** and then applied to both training and test sets to prevent data leakage. The final split and scaling outputs confirm:

- **Train set:** (48855, 128)
- **Test set:** (12214, 128)
  Stratified splitting was used so the train/test class proportions remain consistent with the original distribution.

Section 4: Dataset Splitting

After preprocessing, the dataset was split into **training** and **testing** sets using an **80:20** ratio. A **stratified split** was used (stratify=y) so that both subsets keep approximately the same edible/poisonous class proportions as the full dataset. This is important because the target classes are not perfectly balanced. Figure 9 further confirms that stratified splitting preserved the edible and poisonous class proportions across the training and test sets. The final matrices after encoding contained **128 features**, and the split produced **48,855** training samples and **12,214** testing samples. The training set contained **p=27,110** and **e=21,745**, while the test set contained **p=6,778** and **e=5,436**, confirming that stratification preserved the class distribution. This approach matches the idea in the example report that most data is reserved for learning patterns while a held-out test set is kept for unbiased performance evaluation.

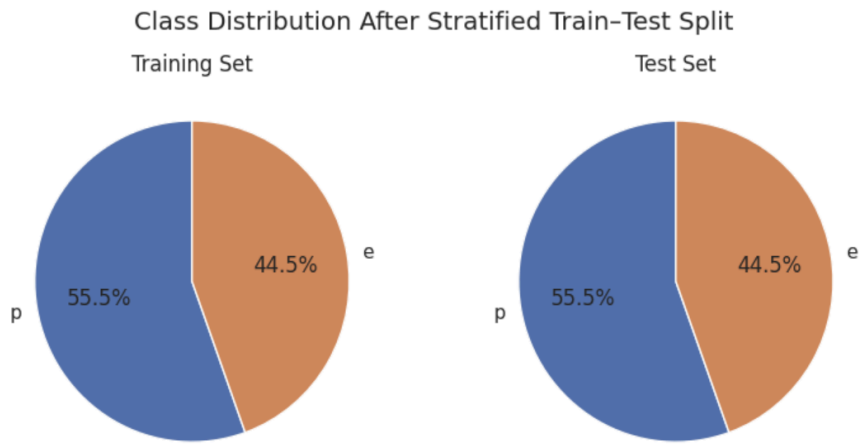


Figure 9: Class Distribution after stratified train-test split

Section 5: Model Training & Testing (Supervised)

Model Selection

To evaluate the mushroom classification task, three supervised learning models were selected: **K-Nearest Neighbors (KNN)**, **Decision Tree**, and a **Neural Network (MLP)**. These models were chosen to represent different learning paradigms: distance-based (KNN), tree-based (Decision Tree), and non-linear function approximation (Neural Network). This combination allows for a meaningful comparison of how different model families perform on the same dataset.

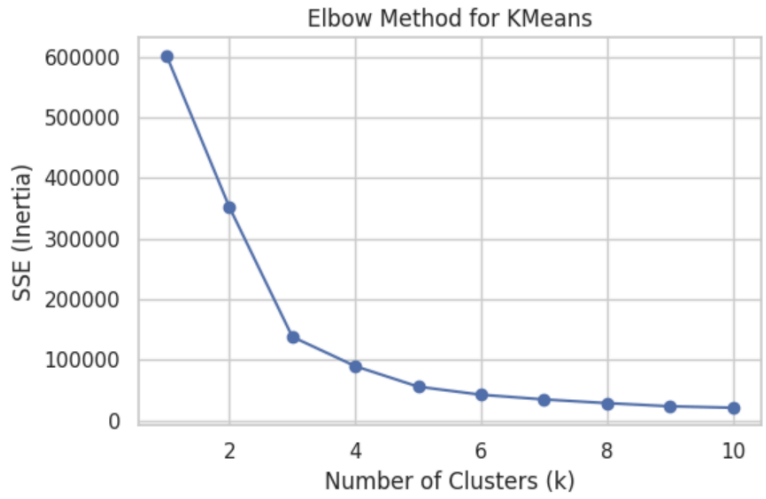
Training Procedure

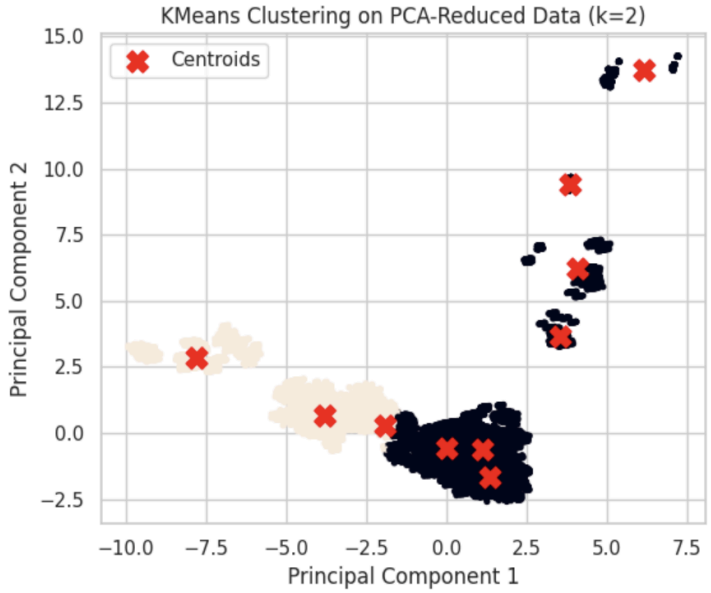
Each model was trained using the **training set (80%)** obtained after preprocessing. The same train–test split was used across all models to ensure a fair comparison. Model training was conducted **both before and after feature scaling**, enabling an analysis of the impact of scaling on model performance.

Evaluation Metrics

Model performance was evaluated on the **test set (20%)** using standard classification metrics: **accuracy, precision, recall, F1-score, confusion matrix, and ROC–AUC**. Because the dataset exhibits a mild class imbalance, reliance on accuracy alone is insufficient; therefore, multiple complementary metrics were reported to assess predictive quality for both edible and poisonous classes.

Unsupervised Learning(K-Mean Clustering):





K-Means clustering was applied to the mushroom dataset by treating the data without using the class label (edible/poisonous). Since the encoded feature space is

high-dimensional, PCA (Principal Component Analysis) was first used to reduce the scaled feature set into 2 principal components so the clusters could be visualized in a 2D plot. Next, the Elbow Method was used by running K-Means for k=1 to 10and plotting the SSE/Inertia for each k. The curve shows a sharp decrease at small k values and then begins to flatten, indicating that after a certain point adding more clusters gives only small improvements. Based on this elbow trend, a small k value was chosen for clustering and visualization. Finally, K-Means was fitted on the PCA-reduced data and the resulting cluster assignments were plotted in the 2D PCA space, with the red "X" markers showing the learned cluster centroids. This visualization demonstrates that K-Means is able to discover natural groupings in the mushroom feature patterns even without supervision, and it provides an unsupervised perspective on how the dataset may separate into distinct structure-based groups.

Section 6: Model Selection & Comparison Analysis

Model Selection:

K-Nearest Neighbors (KNN)

KNN works for this dataset because the dataset is large, features are well-encoded and similar mushrooms behave similarly. The KNN classifier was trained with k = 13 (selected to balance bias and variance). Before feature scaling, KNN already achieved strong performance, reflecting the informative nature of the encoded features. After applying standard scaling, KNN performance remained consistently high, indicating that the dataset's feature representation is well-structured for distance-based learning. Confusion matrices show very few misclassifications in both cases, demonstrating KNN's effectiveness for this task.

Decision Tree

The Decision Tree was chosen because this model captures non-linear relationships and they work well with categorical data. The Decision Tree model was trained without explicit feature scaling, as tree-based methods are invariant to feature scale. The model achieved near-perfect classification accuracy, with confusion matrices showing minimal error. This suggests that the dataset contains strong, easily separable decision rules and that non-linear feature interactions are effectively captured by the tree structure.

Neural Network

A multi-layer perceptron (MLP) neural network with two hidden layers was trained using scaled features. The network consisted of an input layer corresponding to the encoded feature set, two hidden layers with a moderate number of neurons, and a single-node output layer for binary classification. The neural network achieved excellent performance on the test set, with high accuracy and strong precision–recall balance. The ROC curve and AUC further confirm the model's ability to separate edible and poisonous classes.

To compare model performance, KNN (k=13), Decision Tree, and Neural Network were evaluated on the same 80/20 stratified split under two conditions: **before feature scaling** and **after feature scaling**. Performance was compared using accuracy, precision, recall, confusion matrices, and ROC–AUC.

Results Before Feature Scaling:

(a) Accuracy bar chart

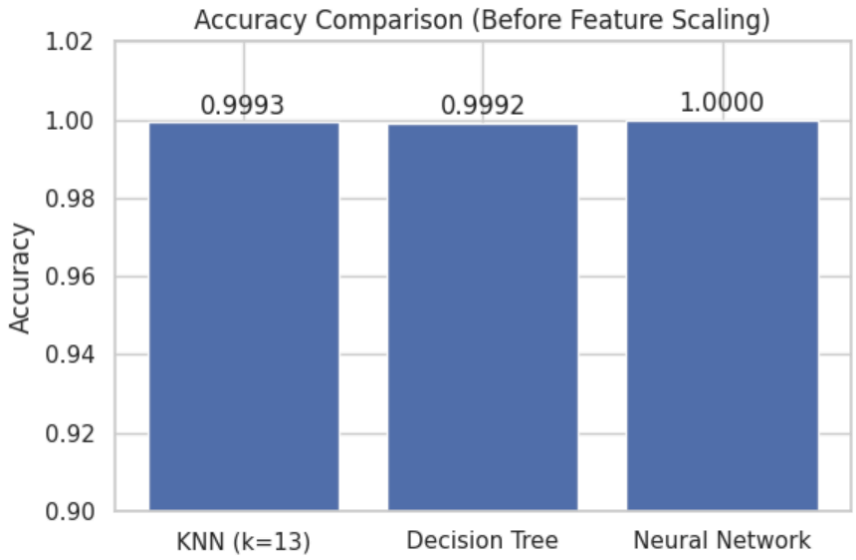


Figure 10: Accuracy comparison of KNN, Decision Tree, and Neural Network before feature scaling

(b) Precision & Recall bar chart

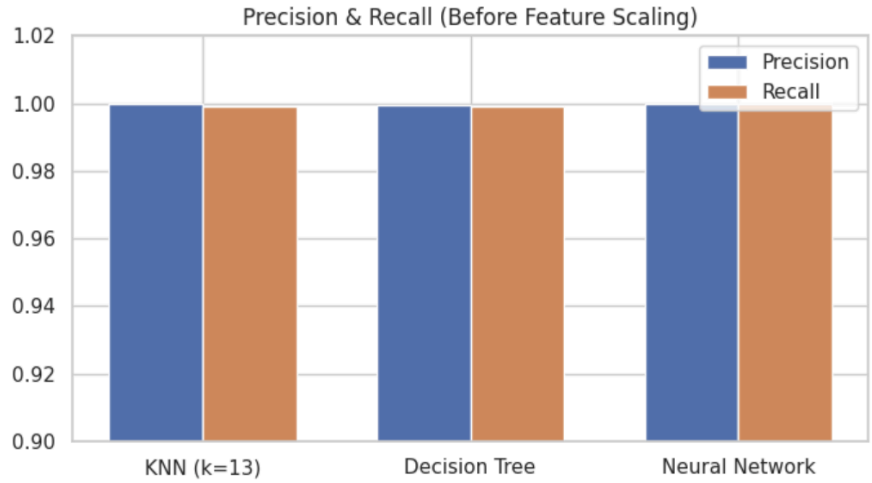
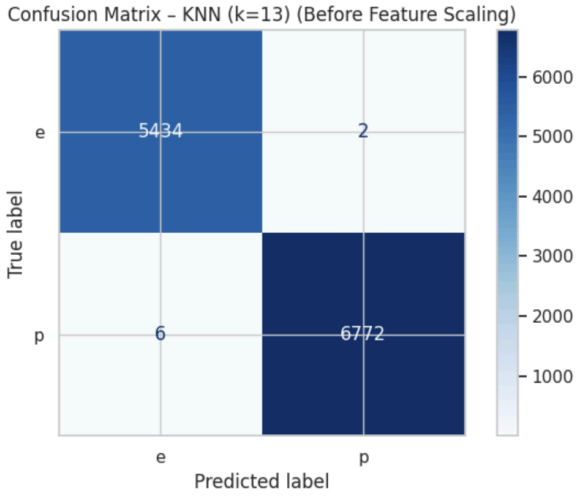
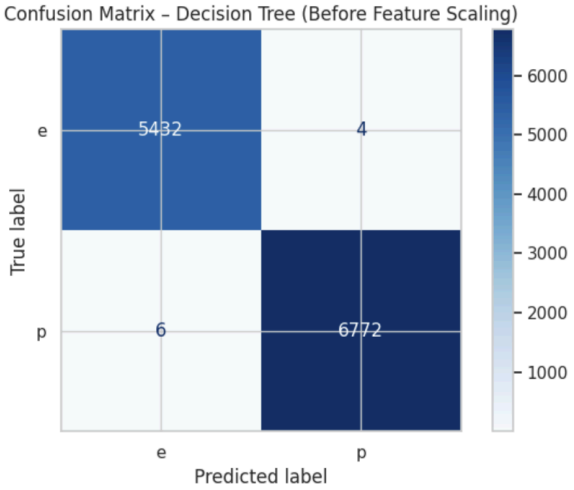


Figure 11: Precision and recall comparison before feature scaling

(c) Confusion matrices





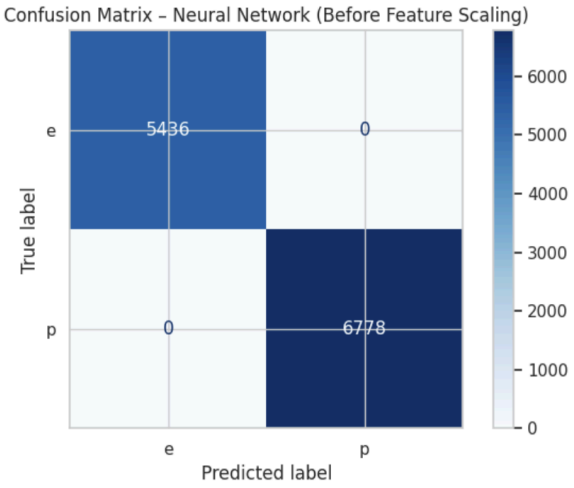


Figure 12: Confusion matrices for all models before feature scaling

(d) ROC curve + AUC

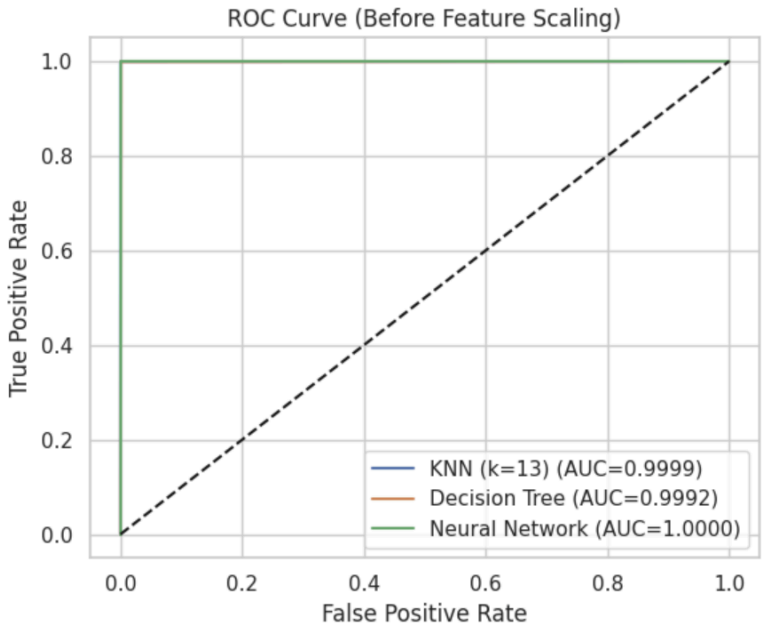


Figure 13: ROC curve comparison and AUC scores before feature scaling

Results After Feature Scaling

(a) Accuracy bar chart

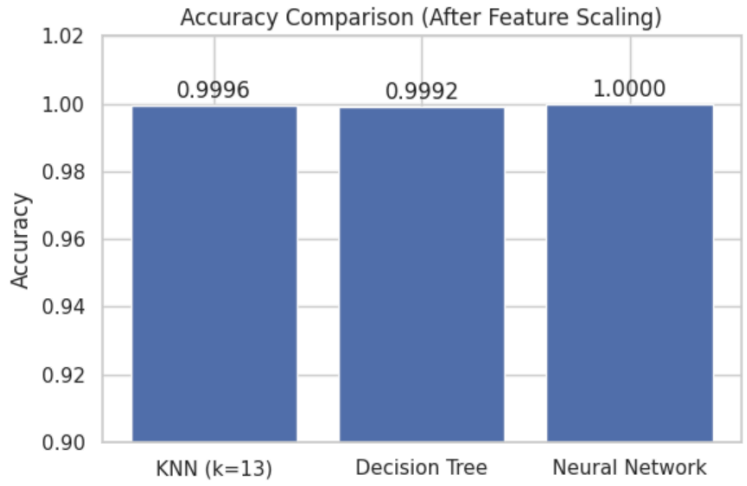


Figure 14: Accuracy comparison of KNN, Decision Tree, and Neural Network after feature scaling

(b) Precision & recall bar chart

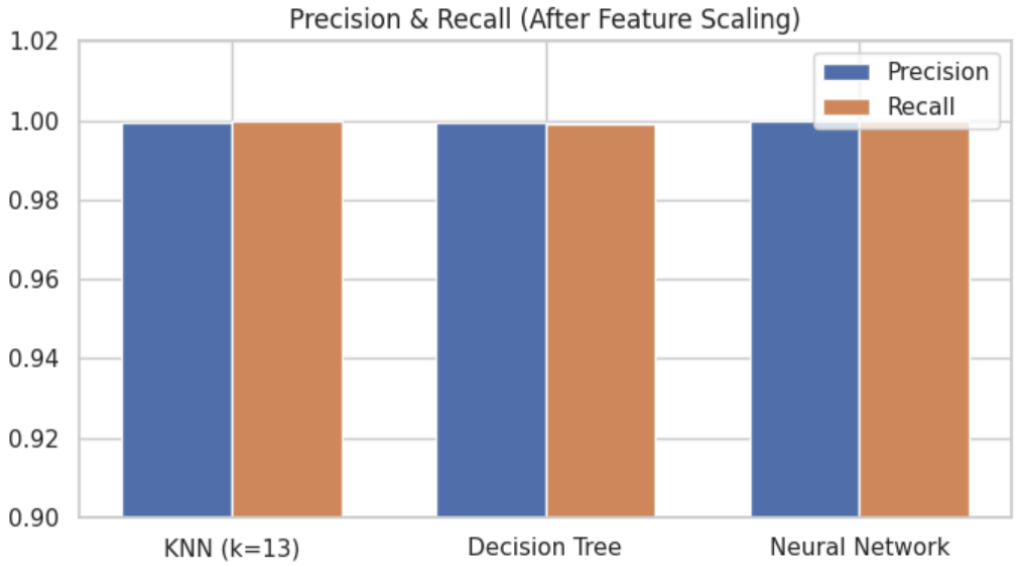
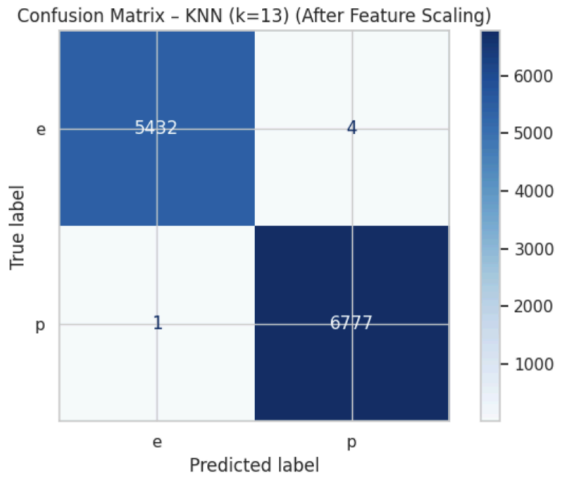
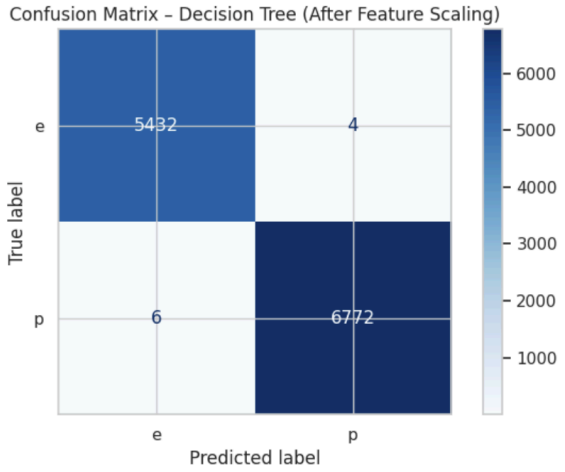


Figure 15: Precision and recall comparison before feature scaling

(c) Confusion matrices





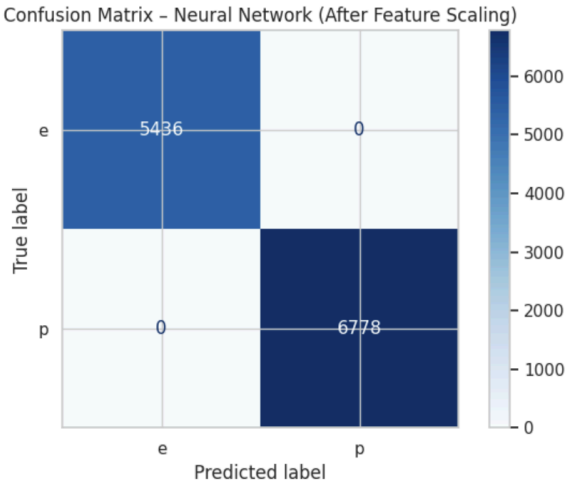


Figure 16: Confusion matrices for all models after feature scaling

(d) ROC curve + AUC

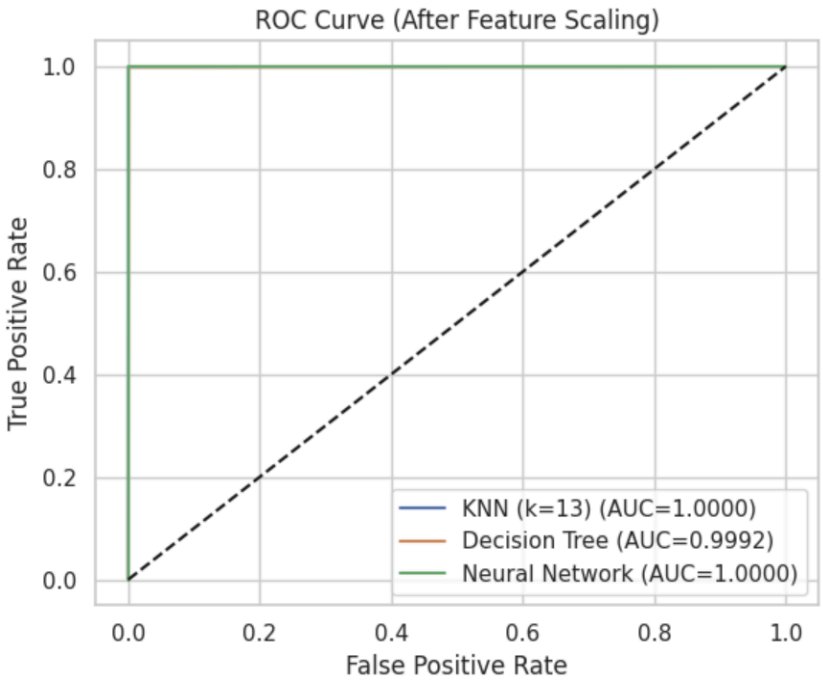


Figure 17: ROC curve comparison and AUC scores before feature scaling

Overall, model performance remains near-perfect in both settings, indicating that the dataset is highly separable after preprocessing and one-hot encoding. Feature scaling had minimal impact on predictive performance across the three models. This outcome is expected because most features are one-hot encoded (binary), and the Decision Tree model is inherently scale-invariant. KNN and the Neural Network showed only negligible differences after scaling, suggesting that nearest-neighbor relationships and learned decision boundaries remain stable under standardization for this dataset. Confusion matrices further confirm this conclusion, as misclassifications remain extremely low in both cases.

Although all three models achieved excellent results, the **Decision Tree** was selected as the final model due to its strong accuracy, stable precision/recall

balance, and interpretability. Unlike KNN and Neural Networks, Decision Trees provide transparent decision rules and require no feature scaling, making them a practical and explainable choice for a safety-relevant classification task.

Section 7: Conclusion

This project developed a complete machine learning pipeline to classify mushrooms as edible or poisonous based on their physical characteristics. Exploratory data analysis helped identify useful numerical and categorical patterns in the dataset, as well as a mild class imbalance. Several preprocessing steps, including handling missing values, one-hot encoding, feature scaling, and stratified train–test splitting, were applied to prepare the data for reliable model training and evaluation.

Three supervised learning models - K-Nearest Neighbors (KNN), Decision Tree, and Neural Network were trained and evaluated using accuracy, precision, recall, confusion matrices, and ROC–AUC. All models achieved very high performance, indicating that the dataset is highly separable. This strong performance is mainly due to the rich set of features, effective preprocessing, and the fact that mushroom edibility depends on combinations of multiple attributes rather than a single feature. Feature scaling had little effect on overall performance because most features became binary after one-hot encoding, and tree-based models are not sensitive to feature scale. Among the tested models, the Decision Tree was selected as the final model due to its high accuracy, stable performance, and interpretability.

Several challenges were faced during the project. A large number of missing values were present in multiple categorical features, which required careful handling to avoid losing important data. One-hot encoding also increased the number of features significantly, making the dataset high-dimensional and more computationally expensive. In addition, selecting suitable hyperparameters, such as the number of neighbors in KNN and the number of clusters in K-Means, required experimentation and validation.